NBA Betting Market Efficiency

An Interactive Analytics Dashboard

FA 550: Data Visualization · Capstone Final Project

Kayo Niebrzydowski
Spring 2026

Data Source: Christopher Treasure · NBA Odds Dataset · Kaggle

Section 1: Executive Summary

Project Title: NBA Betting Market Efficiency Dashboard

Track: Interactive Dashboard / Python + Streamlit

This project is an interactive data dashboard that looks at 16 seasons of NBA betting data to figure out where the sports betting market gets things right and where it doesn't. The goal was to take a massive, messy dataset and turn it into something that an average sports fan or curious analyst could actually use to find patterns and insights.

Key Features/ Technology Used/ Key Findings

- Market Macro Tab: League-wide trends for totals, Over/Under rates, and home-court advantage over time
- Team Edge Tab: A breakdown of which teams cover the spread most often and by how much
- Strategy Simulator Tab: A flat-bet simulation that shows what blindly betting every game would actually cost you
- Playbook Tab: An educational guide that explains every chart and what insight to take from it
- Python, Pandas, NumPy — data cleaning and calculations
- Streamlit — web app framework and deployment
- Plotly Express & Plotly Graph Objects — all visualizations
- Kaggle (Christopher Treasure) — primary data source
- The moneyline market is very well-calibrated — Vegas is accurate at predicting straight-up winners
- The Against-the-Spread (ATS) market has more gaps — teams like the Thunder and Celtics have historically beaten the spread at above-breakeven rates
- Blind betting all games loses about −1,824 units over 16 seasons, which lines up perfectly with the expected −4.5% sportsbook tax

Who Benefits

This dashboard is useful for sports bettors trying to find an edge, sports analysts studying market behavior, and anyone in a data visualization or sports analytics course who wants a real-world example of working with large, messy datasets.

16	37,104	9	4
Seasons of Data	Raw Data Rows	Visualizations	Dashboard Tabs

Section 2: Project Goals & Context

The Problem

Sports betting has gotten way more popular over the last few years, especially since the U.S. Supreme Court opened the door for states to legalize it. But most of the data and tools out there are either super basic or locked behind expensive subscriptions. The question I wanted to answer was simple: is there actually a way to beat the sportsbook over a long period of time, or does the house always win? And how has the market evolved over the years?

This matters because millions of people bet on NBA games without really understanding how the odds are structured or what history says about certain betting strategies. A well-built, accessible dashboard that answers these questions publicly can be genuinely useful. Especially with the new generation doing such, this can be a key insight for pros and cons to observe.

Project Objectives

- Build an interactive dashboard that analyzes real NBA betting data across 16 seasons
- Show whether the Vegas betting market is actually "efficient" — meaning whether the odds accurately reflect reality
- Identify which teams and which types of bets have historically offered the best or worst value
- Make a tool that is easy enough for a non-expert to use, but detailed enough to interest serious analysts

Target Audience

My main audience is sports fans who are interested in betting but don't have a strong analytics background. They know what a point spread is, but they probably don't know how to analyze 16 years of data to see if the spread is being set correctly. The dashboard gives them that analysis without requiring any coding knowledge. So, the charts are detailed enough that a more experienced analyst would still find value in them as it is almost an all audience tool.

Why I Chose This Project

I've always been interested in sports analytics, and sports betting is one of the most data-rich environments out there. I personally live for data and love reading into it. So, every game has spreads, totals, moneylines, and outcomes — all of which can be compared against each other. I also liked the challenge of presenting something that feels a little intimidating (betting math, probability, large datasets) in a way that's clean and readable. This felt like the perfect capstone project because it combines real data, real decision-making, a polished interactive interface, and a way of just observing all of this into a simple guided screen to observe such data.

Section 3: Data Preparation Process

Data Source

The dataset I used is the NBA Odds Dataset published by Christopher Treasure on Kaggle. It contains every NBA game from October 2007 through January 2023, with betting odds information for each game. The raw file is oddsData.csv with 37,104 rows and 12 columns. The key columns include the date, teams involved, the point spread, the moneyline, the Over/Under total, and the actual final scores.

Data Quality Assessment

The dataset was actually pretty clean coming out of Kaggle, which made my job easier. The main issue I ran into was structural rather than dirty data: every game appeared twice, once from each team's perspective. This meant that any league-wide calculation I ran initially gave me doubled results. Beyond that, the moneyline odds required conversion from American odds format to implied probability percentages, which involved some math that isn't totally intuitive.

Cleaning and Transformation Steps

Here is the step-by-step pipeline I used to get the data ready:

1. Loaded the CSV into Python using Pandas and did an initial inspection of column types and null counts.
2. Deduplicated game records — created two versions of the dataset: a full version for team-level analysis, and a single-row-per-game version for league-wide metrics.
3. Applied a team name mapping to standardize team names (e.g., mapping "New Jersey" → "Nets" and "Seattle" → "SuperSonics").
4. Calculated derived columns: actual margin (score minus opponent score), ATS result (Win/Loss/Push), margin of cover, Over/Under result, and implied win probability from moneyline.
5. Applied no-vig normalization to the implied probabilities so that each game's two implied probabilities summed to exactly 100%, removing the sportsbook's built-in margin.
6. Verified zero null values in all primary columns before loading into the dashboard.

Tools Used
Python · Pandas · NumPy · Streamlit @st.cache_data for performance optimization

Section 4: Design Decisions & Rationale

This was probably the section I thought about the most. Every chart I built, I asked myself: is this actually showing something useful, or does it just look like data? Here's how I thought through the major design choices.

Overall Theme and Color Palette

I went with a dark navy theme with amber and cyan accents. The reasoning was pretty practical: a lot of sports betting and analytics platforms use dark themes because they're easier to read for long periods and they give a more "professional analytics" feel. The amber color draws the eye to the most important elements — key metrics, breakeven lines, highlighted teams — and the cyan works great for individual data points on scatter plots because it pops against the dark background without being harsh.

Visualization Selection

- **Scatter Plot (Expected vs. Actual Totals):** I needed something that could show 18,000+ individual game records at once and still be readable. A scatter plot was the only real option for that. The dashed red "perfect calibration" line gives the viewer an instant reference point — dots above the line went Over, dots below went Under.

- **Stacked Area Chart (Market Outcome % by Season):** I wanted to show how the Over/Under split has changed across time. A stacked area chart is ideal for this because it shows proportions adding up to 100% and makes trends across seasons easy to see at a glance.

- **Line Chart (Home-Court Decay):** A simple line chart was the right call for showing one metric changing over time. I didn't want anything fancy here — just a clean amber line showing the average home-team margin by season. The horizontal dotted line at zero gives the "no advantage" reference.

- **Horizontal Bar Charts (ATS Leaderboard + Margin of Cover):** I chose horizontal over vertical bars because team names are long and I didn't want to angle the text. Sorted by value, these charts let you instantly see the top and bottom performers. The gold dashed line at 52.4% on the ATS chart is critical — that's the breakeven point, and it's the whole story of Tab 2.

- **Heatmap (Season-by-Season Consistency):** I replaced my original donut chart with this because the home/away split was boring. The heatmap gives you 16 seasons of data for 15 teams in one grid. Green cells = profitable ATS season, red cells = losing season. It's the best chart in the dashboard for spotting which teams are consistent versus fluky.

- **Area Chart (Equity Curve):** An equity curve is the standard way to show bankroll performance over time in trading and betting. The fill under the line reinforces the direction visually — red fill going down hurts to look at, which is exactly what I wanted when showing the blind-betting result.

- **Bubble Scatter (Moneyline Calibration):** I needed to show both the implied win probability and the actual win rate for grouped probability buckets, plus indicate how many games were in each bucket. A bubble scatter handles all three dimensions cleanly — X axis, Y axis, and bubble size.

Layout and Tab Structure

My original plan had everything on one page. During development it got overwhelming fast, so I moved to a four-tab structure. Tab 1 is the big picture. Tab 2 zooms into individual teams. Tab 3 is the strategy simulator where users can test specific filters. Tab 4 is the explainer for people who are new to betting terminology. This flow made sense because it goes from broad to specific to actionable to educational — a natural progression.

Interactivity Design

The three sidebar filters — Season Range slider, Team multi-select, and Venue radio button — update all charts in real time. I chose these because they answer the most natural questions a user would have: "What about just the last five years?" or "What if I only look at the Thunder?" or "Does this change if I only count home games?" Every filter works across all four tabs, so users aren't lost switching between views.

Section 5: Technical Implementation

Technology Stack

- **Python 3.11:** Core language for all data processing, logic, and app code.
- **Pandas + NumPy:** Data cleaning, transformation, and calculated columns.
- **Streamlit:** Web app framework. Chosen because it lets you build a polished interactive dashboard entirely in Python without needing any JavaScript or HTML.
- **Plotly Express + Graph Objects:** All charting. Plotly was the right pick because it handles hover interactions, click events, and responsive sizing natively.
- **Streamlit Cloud:** Free hosting. The app is deployed at a public URL with no login required.

Project Structure

The app is a single main Python file (app.py) that handles all four tabs. The data lives in a separate oddsData.csv file. The @st.cache_data decorator on the data loading function is what makes everything fast — the 37,000-row dataset gets processed once at startup and then stored in memory, so filter changes don't cause full reloads.

Key Technical Implementations

The most interesting technical piece was the deduplication logic. Since every NBA game has two rows (one per team), I had to create two versions of the DataFrame: one full version for team-level analysis that needs both rows, and one deduplicated version for league-wide calculations that only keeps the home team's row per game. I used a combination of the date and the home team identifier to generate a unique game key for deduplication.

The implied probability conversion was another tricky part. American moneyline odds require a different formula depending on whether they're positive or negative. After converting both sides of every game, I applied no-vig normalization so the two probabilities summed to 100%, which is necessary for the calibration chart to make sense.

Sample Code: Implied Probability Conversion

```
def calc_implied_prob(ml):    if pd.isna(ml): return np.nan    if ml < 0: return abs(ml) / (abs(ml) + 100)    else: return 100 / (ml + 100)df['implied_win_prob'] = df['moneyLine'].apply(calc_implied_prob)
```

Section 6: Key Findings & Insights

Finding 1: The Moneyline Market is Highly Accurate

The Moneyline Calibration chart shows that Vegas is really good at setting win probabilities. The bubble chart points fall almost exactly on the perfect calibration line, which means if Vegas says a team has a 70% chance of winning, they actually win about 70% of the time. This is one of the strongest signs that the straight-up winner market is efficient and hard to beat.

Finding 2: The ATS Market Has Exploitable Gaps

The Against-the-Spread (ATS) Leaderboard tells a different story. While most teams cluster around the 50% mark, the Oklahoma City Thunder posted the highest ATS win rate across the dataset at 53.2%, and the Boston Celtics had the widest positive average margin of cover at +0.70 points. These might sound like small differences, but over hundreds of games they add up to a real edge over the sportsbook's 52.4% breakeven threshold.

Finding 3: Blind Betting Guarantees a Loss

The Strategy Simulator's Cumulative Equity Curve makes this undeniable. Betting one unit on every single game across all 16 seasons results in a net loss of approximately −1,824 units and a return on investment of −4.5%. This is exactly in line with what theory predicts given standard −110 sportsbook odds. The math is the math.

Finding 4: Home-Court Advantage is Declining

The Home-Court Decay line chart shows a clear long-term downward trend in the average home team win margin. It spiked during the COVID bubble seasons (where all games were played at neutral sites) but has not fully recovered since. This is meaningful for bettors because Vegas may still be pricing in a home-court premium that the data no longer supports as strongly as it once did.

Limitations

- The dataset ends in January 2023 and does not include recent seasons, so current market conditions may differ
- The analysis treats all teams across all eras equally — a dynasty era team (Warriors 2015-2019) skews the team-level data
- The simulation uses flat-unit betting and does not account for variable bet sizing, which is how most sophisticated bettors actually operate
- No injury data is included, which is one of the biggest factors in real-world betting line movement

Section 7: Challenges & Solutions

Here are the four biggest obstacles I ran into and how I worked through them:

Challenge	What Happened	How I Fixed It	AI Role
Duplicate Game Rows	Every game appeared twice (one row per team), doubling all league-wide calculations	Created two DataFrames: one full set for team analysis, one deduplicated by unique game key for league totals	Gemini suggested the groupby approach after I pasted a sample of the data structure
Implied Prob Math	Raw moneyline conversion produced probabilities over 100% due to the sportsbook vig	Applied no-vig normalization so both sides of each game summed to exactly 100%	Gemini explained the normalization step and helped write the Pandas function
Slow Load Times	37,000+ rows caused the dashboard to re-process everything on every filter change	Used @st.cache_data to load and clean data once at startup and hold it in memory	Gemini diagnosed the missing cache immediately and flagged the 'mutate cached data' bug risk
Visual Clutter	All 9 charts on one page was overwhelming and unfocused	Reorganized into a four-tab structure with a logical flow: big picture → teams → simulator → explainer	Gemini helped brainstorm the tab naming and organizational logic

Section 8: AI Usage Throughout the Project

Tools I Used

- **Google Gemini:** My primary AI assistant throughout the project. Used it for debugging, data pipeline advice, chart layout ideas, and tab organization.
- **Claude (Anthropic):** Used for secondary input on specific data transformation questions, particularly around the deduplication logic and polished visual looks.

Where AI Was Most Helpful

Gemini was genuinely useful for the technical debugging side of things. When my dashboard was running slow, I described the problem and Gemini immediately pointed me to @st.cache_data as the fix — it also warned me about a specific Streamlit bug where you can't mutate cached DataFrames in place, which I wouldn't have known about until it caused a weird error later. That kind of proactive advice saved real time.

For the deduplication logic, I pasted a sample of the raw data structure into Gemini and described what I needed — a league-level dataset where each game only appears once. Gemini suggested the groupby approach with a unique game key built from date and home team name. I verified the row counts before and after to confirm it worked correctly.

On the math side, Gemini walked me through the no-vig normalization step for implied probability conversion and then wrote the Pandas implementation, which I tested against several manually calculated examples before trusting it in the full dataset.

> **Sample AI Interaction (Summarized)**
> My Prompt: "I have a Streamlit dashboard with 37,000+ rows and it's re-running all calculations every time a slider moves. What's the fix?"Gemini Response: Apply @st.cache_data to your data loading function. This stores the processed DataFrame in memory so only the filtering logic re-runs on interactions, not the full data pipeline. Also avoid mutating the cached DataFrame in place — copy it first.Outcome: Load times dropped from ~10 seconds to under 3 seconds across all filter combinations.

Where AI Struggled

AI was less helpful when it came to actual design judgment. When I asked Gemini to pick between a box-and-whisker plot and a calibration scatter for my third tab, it gave me a generic answer that wasn't wrong but also wasn't very useful — something like "both could work depending on your goals." I had to make that call myself based on what the data actually

showed. The box plots turned out to be visually boring because the distributions were almost identical across teams, which Gemini couldn't have known without seeing the data.

Prompting Strategies That Worked

- Being specific: Instead of asking "how do I clean my data," I said "I have a 37,000-row DataFrame where each game has two rows, one per team. Here's a sample. How do I create a single-row-per-game version?"
- Showing context: Pasting a sample of the actual data structure got much better answers than describing it in the abstract
- Asking for verification steps: After AI wrote code, I asked it to also tell me how to verify the output was correct — that got me to the row count check method

Section 9: User Testing & Feedback

I ran informal usability testing with three people before finalizing the dashboard. Here's what I found:

Testing Approach

I shared the Streamlit link with five testers. One was a friend who actively uses sports betting apps and has a background in predictive picks. Two were classmates from unrelated programs who had no betting background. I asked each person to spend about 5 minutes on the dashboard and tell me what they understood, what confused them, and what they wished was there. And my roommate who has some knowledge and general input on the dashboard visuals and what should be changed or added, to make it feel and look improved. As the fifth didn't exactly help much and she just said that it looked fine which happened with testers.

Key Feedback

- **The sports-betting-experienced tester:** Loved the ATS leaderboard and the equity curve. Suggested adding a second line to the equity curve showing a profitable filtered strategy next to the baseline. Said the calibration chart was the most interesting thing he'd seen in a while because it confirmed what experienced bettors already knew intuitively.

- **The non-betting testers:** Two said the terminology was a barrier. They didn't know what ATS meant, what the juice was, or why 52.4% was the breakeven. This directly led me to build Tab 4 — The Playbook — which explains every chart and every piece of terminology in plain language. One gave insight on a more polished and bolder look of making the visuals pop which I added lines around the graphs, as for the fifth well not much feedback was given by them.

Changes Made

- Added the entire Playbook tab (Tab 4) as a result of non-expert feedback
- Added plain-language sub-labels below each KPI metric card explaining what the number means
- Added the Baseline Warning alert on the Strategy Simulator explaining why the red curve is supposed to go down
- Added a box line outline and sharper colors to define the visuals more.

Section 10: Reflection & Future Directions

What I Learned

This project changed how I think about data visualization. Before this, I thought the goal was just to make charts that look good. Now I understand that the real goal is to make charts that answer specific questions — and that means spending as much time on what to build as on how to build it. The moment where this clicked for me was when I threw out the box-and-whisker plot and replaced it with the calibration chart. The box plot looked fine. The calibration chart actually said something.

I also got a lot better at working with AI tools in a practical way. The key lesson was specificity. The more context I gave, the better the answers got. Vague questions got vague answers. Specific questions with sample data got specific, usable answers.

How the Project Evolved from the Proposal

- Added a fourth tab (Playbook) that was not in the original plan — this turned out to be one of the most valuable additions
- Replaced the box-and-whisker spread variance chart with the Moneyline Calibration bubble scatter — more visually interesting and analytically meaningful
- Replaced a planned donut chart (home vs. away ATS rates) with the season-by-season heatmap — the donut showed nearly identical splits that weren't worth visualizing

What I Would Do Differently

If I started this over, I would plan the tab structure upfront instead of reorganizing mid-build. A lot of time went into rearranging code after the fact that would have been saved with a better initial architecture. I'd also spend more time upfront thinking about what "done" looks like for each chart before writing a single line of code.

Future Enhancements

- Add a second equity curve line to the simulator showing what a filtered/targeted strategy looks like versus the blind betting baseline
- Integrate current season data so the dashboard updates automatically and isn't limited to the 2008-2023 window
- Add player-level data (injuries, rest days) to explain line movement rather than just showing outcomes
- Build a "Find Your Edge" feature that automatically surfaces the top three statistically significant patterns for whatever team or season a user is looking at

Broader Applications

The methodology I used here — taking a large, publicly available dataset and surfacing market inefficiencies through interactive visualization — applies to a lot of other domains. Financial markets, election polling, real estate pricing, and supply chain forecasting all have the same basic structure: a prediction, an outcome, and a gap worth studying. The dashboard architecture I built could be adapted to any of those problems with a different dataset.

References

- Treasure, C. (2023). NBA Odds Dataset [Data set]. Kaggle. https://www.kaggle.com/datasets/christophertreasure
- Streamlit Documentation. (2024). st.cache_data. https://docs.streamlit.io
- Plotly Technologies Inc. (2024). Plotly Python Open Source Graphing Library. https://plotly.com/python/
- Massey, K. (1997). Statistical models applied to the rating of sports teams. The Mathematics of Sports, 1(1), 1–10.

AI Disclosure Statement

Academic Integrity Pledge

I pledge in my honor that I have not given or received any unauthorized assistance on this assignment/examination. I further pledge that I have not copied any material from a book, article, the Internet or any other source except where I have expressly cited the source. I have documented all AI tool usage including prompts and tool selection rationale.

Signature: **Kayo Niebrzydowski** Date: **May 2026**